

Signify

Project Evaluation Guide

Dear Reviewer,

Welcome to the codebase for **Signify**, our real-time American Sign Language (ASL) avatar. This document provides a high-level overview of our architecture, explains how the code is structured, and points out the key engineering decisions we are most proud of.

1. The Big Picture (How It Works)

Signify translates spoken English into ASL animation in real-time. Data flows sequentially through independent, non-blocking stages:

1. **Audio Capture:** Intercepts system audio (e.g., a YouTube video) or a microphone.
2. **Transcription:** Converts the spoken words into English text.
3. **Translation:** Restructures the English text into ASL grammar (e.g., removing stop words, applying Time-Topic-Comment ordering).
4. **Animation Engine:** Retrieves the movement coordinates for the translated words.
5. **Rendering:** Draws the avatar and outputs it to the user's screen.

2. Code Structure (Where to Look)

Our repository is divided into logical components:

- `main.py` : The central hub that boots up the system and connects all modules.
- `audio/` , `core/` : The brain of the app (capturing audio, translating text, and animating).
- `aws/` : Our custom cloud data factory scripts (used offline to build the dictionary).
- `output/` : Manages our rendering modes (Local Window, Virtual Camera, WebSockets).

- `website/` : Contains our Landing Page and the Smart TV app source code.
- `chrome-extension/` : The source code for our browser extension.

3. Key Engineering Highlights

We focused heavily on **ultra-low latency** and **clever architecture**:

Zero AI at Runtime (The AWS Pipeline)

Running AI locally is too slow for a seamless real-time conversation. Instead, we built an offline AWS pipeline (`aws/cloud_pipeline.py`) that extracts body coordinates from ASL videos and saves them as tiny files. At runtime, our app simply fetches these from Amazon S3. This eliminates heavy computation, making our app incredibly fast.

Queue-Decoupled Multi-Threading

Every stage (listening, translating, animating) runs on its own independent thread. They communicate exclusively through Python `Queues` . If transcription takes an extra second, the animation never freezes.

Dynamic Frame Interpolation

To save network bandwidth, we only fetch 30 keyframes per sign from the cloud. Our Animator mathematically calculates smooth intermediate frames on the fly, resulting in a buttery-smooth 60 FPS animation.

4. The Signify Ecosystem (What We Built)

We didn't just build a local desktop script; we built a complete product ecosystem to ensure accessibility across all platforms:

- **Landing Page:** We built a complete product website ([View Landing Page](#)). From here, users can seamlessly download the standalone Desktop App and the Chrome Extension.
- **Zoom & Video Conferencing:** Using OBS Virtual Camera integration (`output/virtual_cam.py`), our desktop app can act as a native webcam. Users can select "Signify" as their camera in Zoom, Teams, or Google Meet to project the ASL avatar directly into their meetings.

- **Chrome Extension:** We developed a browser extension ([chrome-extension/](#)) that overlays the Signify avatar directly on top of streaming websites like YouTube and Twitch, providing real-time translation for web video.
- **LG Smart TV App & Smart Delay:** We created a webOS TV application ([website/lg_tv_app/](#)) that receives the avatar broadcast over WebSockets. Because processing audio into sign language inherently takes a moment, we implemented a clever **delay / buffering mechanism** on the TV app. This allows the system to perfectly synchronize the on-screen ASL avatar with the broadcasted video, creating a seamless viewing experience without awkward mismatched timing.